



ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA DE SISTEMAS

INGENIERÍA DE SOFTWARE

TAREA 06

PRIMER AVANCE TÉCNICO-ARQUITECTÓNICO DEL SITIO WEB ACCESIBLE

Asignatura: Usabilidad y Accesibilidad (ISWD732)

Curso: GR3SW

Tema:

**AVANCE DEL PROYECTO VITAPREVENT:
PLATAFORMA WEB ACCESIBLE DE SERVICIOS
PÚBLICOS DE SALUD**

Grupo 6: Javier Quilumba

Jonathan Tipán

Erick Costa

Docente: Ing. Pablo Andrés Saá P.

Fecha: 28/06/2026



Índice

1. Introducción	1
2. Descripción General del Sitio	1
2.1. Tema del sitio	1
2.2. Público objetivo	2
2.3. Objetivo comunicacional y funcional	2
3. Justificación del Proyecto	2
4. Objetivo General	3
5. Objetivos Específicos	3
6. Organización del Trabajo	4
7. Visión Técnica General del Proyecto	4
8. Mapa de Páginas o Secciones	5
9. Sistema de Rutas Propuesto	6
10. Tecnologías Utilizadas	6
11. Arquitectura Funcional del Sistema	7
12. Modelos de Datos Propuestos en Firestore	8
13. Componentes Reutilizables	8
14. Requisitos del Sitio Considerados en Este Avance	9
14.1. Estructura y navegación	9
14.2. Contenido	9
14.3. Formularios	10
14.4. Interactividad	10
14.5. Accesibilidad visual	10
15. Estrategia de Accesibilidad	10
16. Aplicación Inicial del Ecosistema W3C/WAI	11
16.1. WCAG 2.2	11
16.2. WAI-ARIA	11
16.3. ATAG	12

16.4. UAAG	12
17. Metodología de Evaluación Prevista	12
18. Matriz Inicial de Criterios WCAG 2.2 AA	13
19. Evidencias Previstas	17
20. Declaración de Conformidad del Avance	18
21. Alcance del Primer Avance	18
22. Limitaciones del Avance	19
23. Próximas Actividades	19
24. Conclusiones	20
25. Recomendaciones	20
Referencias	21

1. Introducción

El presente informe corresponde al avance técnico-arquitectónico del proyecto **SaludEC**, una plataforma web de servicios públicos de salud del Ecuador, desarrollada para la asignatura de Usabilidad y Accesibilidad (ISWD732).

En esta etapa se documenta el estado actual del proyecto: su estructura, módulos implementados, tecnologías seleccionadas, decisiones de accesibilidad adoptadas y los criterios WCAG 2.2 aplicados hasta la fecha. El sitio se encuentra funcional y desplegado en producción en vitaprevent-b2e34.web.app, con todas sus páginas principales operativas.

Como grupo, buscamos que SaludEC no sea solamente una página informativa, sino una plataforma web funcional, organizada y accesible, orientada a ciudadanos ecuatorianos que deseen conocer y acceder a los servicios públicos de salud sin barreras de ningún tipo. Por esta razón, el proyecto fue planificado e implementado desde el inicio considerando navegación clara, formularios accesibles, contenido comprensible, componentes interactivos y compatibilidad con distintos usuarios y dispositivos.

2. Descripción General del Sitio

2.1 Tema del sitio

El tema seleccionado es el desarrollo de una plataforma web accesible de **servicios públicos de salud**.

Ficha del sitio

Nombre:	SaludEC
Eslogan:	“Tu salud, un derecho garantizado”
Enfoque:	Servicios públicos de salud del Ecuador
URL:	https://vitaprevent-b2e34.web.app
Repositorio:	https://github.com/zeuspyEC/SaludEC

SaludEC es un portal digital orientado a promover el conocimiento y acceso a los servicios públicos de salud del Ecuador, con contenido sobre Atención Primaria (centros del MSP e IESS), Vacunación (Programa Ampliado de Inmunizaciones), Salud Mental (servicios y líneas de crisis del MSP) y Emergencias (ECU 911 y SAMU). El sitio está diseñado para facilitar el acceso a información clara sobre cómo usar el sistema de salud pública, con cifras oficiales de fuentes como el MSP, IESS, ECU 911 y OPS Ecuador.

2.2 Público objetivo

La plataforma está dirigida a un público amplio, ya que el acceso a los servicios públicos de salud es un tema relevante para personas de distintas edades y condiciones. Se toma en cuenta:

- Niños, adolescentes, adultos y adultos mayores.
- Personas con discapacidad visual (usuarios de lectores de pantalla NVDA, VoiceOver).
- Personas con discapacidad motriz (usuarios de teclado exclusivamente).
- Personas con dificultades cognitivas leves.
- Personas con bajo dominio tecnológico.
- Personas que acceden desde dispositivos móviles con conectividad limitada.

Este público justifica la necesidad de que el sitio sea claro, accesible, fácil de navegar y compatible con diferentes formas de interacción.

2.3 Objetivo comunicacional y funcional

El **objetivo comunicacional** de SaludEC es informar y orientar a los ciudadanos ecuatorianos sobre los servicios públicos de salud disponibles, usando un lenguaje sencillo, directo y comprensible, con datos de fuentes oficiales (MSP, IESS, ECU 911, OPS).

El **objetivo funcional** es permitir que el usuario pueda navegar por las cuatro secciones de servicios de salud, consultar artículos y guías, revisar recursos educativos, acceder a noticias de salud pública y enviar mensajes mediante un formulario de contacto accesible.

3. Justificación del Proyecto

Los portales institucionales de salud en Ecuador frecuentemente presentan barreras de accesibilidad que dificultan su uso por parte de personas con discapacidades, adultos mayores o personas con bajo dominio tecnológico. Formularios sin etiquetas, imágenes sin texto alternativo, navegación imposible por teclado y contraste insuficiente son problemas comunes que violan los derechos de acceso a la información.

SaludEC responde a esta problemática construyendo desde el inicio una plataforma que cumpla los estándares internacionales WCAG 2.2 Nivel AA. No se trata únicamente de diseñar una página visualmente agradable, sino de construir una plataforma que pueda utilizarse con teclado, lector de pantalla, dispositivos móviles y distintos navegadores,

garantizando que ningún ciudadano quede excluido por razones tecnológicas.

Desde la asignatura, el proyecto permite aplicar de manera práctica los principios de usabilidad y accesibilidad web, validados mediante herramientas automáticas (axe DevTools, Lighthouse), revisión manual y pruebas con tecnología asistiva (NVDA).

4. Objetivo General

Desarrollar una plataforma web funcional, usable y accesible de servicios públicos de salud del Ecuador, aplicando criterios de accesibilidad web basados en **WCAG 2.2 Nivel AA**, con el fin de ofrecer una experiencia clara, organizada y compatible con distintos usuarios, dispositivos y tecnologías asistivas.

5. Objetivos Específicos

1. Diseñar una plataforma web con estructura semántica clara, navegación coherente y contenido organizado por módulos de servicios públicos de salud.
2. Implementar páginas funcionales sobre Atención Primaria (MSP/IESS), Vacunación (PAI), Salud Mental, Emergencias (ECU 911/SAMU), Biblioteca Digital, Noticias, Contacto y Nosotros.
3. Utilizar React 19, Vite 6, React Router v6 y Firebase para construir una aplicación moderna, modular y desplegable en la nube.
4. Aplicar HTML semántico, CSS accesible, JavaScript no obstructivo y WAI-ARIA únicamente cuando sea necesario.
5. Incluir componentes interactivos accesibles: Accordion (FAQs), Modal, filtros de categoría, paginación y calculadora de IMC con validación.
6. Crear un formulario de contacto accesible con etiquetas visibles, instrucciones claras, validación y mensajes de error comprensibles anunciados mediante `aria-live`.
7. Evaluar el sitio con herramientas automáticas (axe DevTools, Lighthouse), revisión manual y prueba con NVDA, documentando los resultados en este informe.

6. Organización del Trabajo

Cuadro 1: Distribución de roles del equipo SaludEC

Integrante	Rol asignado	Responsabilidades principales
Erick Costa	Coordinador del proyecto y evaluador de accesibilidad	Coordinar avances, organizar tareas, implementar arquitectura base y frontend principal, revisar conformidad WCAG.
Jonathan Tipán	Desarrollador frontend y diseñador de interfaz	Implementar componentes, páginas, sistema de tokens CSS, navegación y criterios de accesibilidad desde el frontend.
Javier Quilumba	Diseñador de interfaz y redactor del informe	Propuesta visual, organización de contenidos, evaluación WCAG con herramientas y redacción del informe técnico.

Aunque cada integrante tiene un rol principal, las actividades se trabajan de forma conjunta. El historial de commits en GitHub refleja la contribución alternada de cada integrante con su identidad correcta.

7. Visión Técnica General del Proyecto

SaludEC ha sido desarrollado como una **SPA (Single Page Application)** con React 19 y Vite 6. Para la navegación interna se usa React Router v6 con `createBrowserRouter` y code splitting por ruta (lazy loading), lo que permite cargar únicamente el código necesario para cada página y optimizar el tiempo de carga inicial.

El proyecto utiliza **Firestore** como backend:

- **Firestore:** Almacena artículos, noticias, recursos y mensajes de contacto.
- **Authentication:** Protege el panel administrativo con email/password.
- **Hosting:** Publica el sitio con CDN global y HTTPS automático.

El código fuente se gestiona en github.com/zeuspyEC/SaludEC, con despliegue automático a Firebase Hosting mediante GitHub Actions en cada push a la rama principal.

La accesibilidad es un eje transversal del desarrollo: se usa `eslint-plugin-jsx-a11y` que detecta violaciones WCAG en tiempo de compilación, evitando que errores de accesibilidad lleguen al código desplegado.

8. Mapa de Páginas o Secciones

Cuadro 2: Mapa de páginas del sitio SaludEC

Página / Módulo	Descripción
Inicio	Página principal con cubo 3D interactivo, cifras oficiales de salud Ecuador (MSP, IESS, ECU 911), accesos a los 4 módulos y noticias recientes.
Atención Primaria	Guías sobre centros del MSP e IESS, cómo sacar turnos, qué incluye el primer nivel, calculadora de IMC y FAQs.
Vacunación	Esquema nacional de vacunación (PAI Ecuador), beneficios, artículos sobre campañas del MSP y FAQs.
Salud Mental	Señales de alerta, recursos de salud mental públicos, líneas de crisis (disponibles 24/7), artículos y FAQs.
Emergencias	ECU 911, SAMU, hospitales de guardia, protocolos de primeros auxilios y FAQs sobre emergencias médicas.
Biblioteca Digital	Recursos filtrados por tipo (PDF, video, guía, infografía, podcast) con texto alternativo y descripción.
Noticias	Feed paginado de noticias de salud pública con imagen, categoría y fecha.
Contacto	Formulario accesible con validación, mensajes de error en tiempo real y confirmación al enviar.
Nosotros	Presentación del propósito del sitio, enfoque de accesibilidad y equipo.
Panel Administrativo	Módulo protegido con Firebase Auth para gestionar artículos, noticias, recursos y mensajes.
Página 404	Página accesible para rutas inexistentes con enlace de regreso al inicio.

9. Sistema de Rutas Propuesto

Cuadro 3: Rutas implementadas en SaludEC

Ruta	Página
/	Inicio
/atencion-primaria	Atención Primaria de Salud (MSP/IESS)
/atencion-primaria/:slug	Artículo individual de atención primaria
/vacunacion	Vacunación (PAI Ecuador)
/vacunacion/:slug	Artículo individual de vacunación
/salud-mental	Salud Mental
/salud-mental/:slug	Artículo individual de salud mental
/emergencias	Emergencias (ECU 911 / SAMU)
/emergencias/:slug	Artículo individual de emergencias
/biblioteca	Biblioteca Digital
/noticias	Feed de noticias de salud pública
/noticias/:id	Noticia individual
/contacto	Formulario de contacto
/nosotros	Acerca del proyecto y equipo
/admin	Panel administrativo (protegido)
/admin/login	Inicio de sesión del administrador
*	Página 404 accesible

10. Tecnologías Utilizadas

Para el desarrollo del proyecto se utilizaron las siguientes tecnologías:

Cuadro 4: Stack tecnológico de SaludEC

Tecnología	Versión	Propósito
React	19.0.0	Framework UI basado en componentes
Vite	6.3.5	Bundler y servidor de desarrollo
React Router	v6	Enrutamiento de la SPA
Firebase Firestore	v10	Base de datos NoSQL documental
Firebase Auth	v10	Autenticación del panel admin
Firebase Hosting	v10	CDN global y HTTPS automático
CSS personalizado	–	Tokens CSS, Flexbox, Grid
ESLint + jsx-a11y	–	Linting y accesibilidad en compilación
GitHub Actions	–	CI/CD despliegue automático

Se evitó el uso de frameworks visuales como Bootstrap o Material UI para mantener control total sobre la estructura HTML, los estilos y la accesibilidad de cada componente.

11. Arquitectura Funcional del Sistema

La plataforma está organizada en módulos independientes. Esta estructura permite que cada sección pueda desarrollarse, mantenerse o modificarse sin afectar directamente al resto del sitio.

Listing 1: Estructura de directorios de src/

```

1 src/
2   components/
3     ui/      # Button, Badge, Spinner (átomos)
4     layout/  # Navbar, Footer, SkipLink, Breadcrumb, PageWrapper
5     common/  # ArticleCard, FormField, Accordion, Modal,
                SectionHero
6   contexts/  # AuthContext, ToastContext
7   hooks/     # useFocusTrap, useAnnouncer
8   pages/     # Home, Nutricion, ActividadFisica, SaludMental,
9               # Prevencion, Biblioteca, Noticias, Contacto,
                Nosotros, Admin
10  services/   # articulos.service.js, noticias.service.js, recursos
                .service.js
11  styles/     # tokens.css, reset.css, global.css, animations.css

```

```

12  utils/      # validators.js, formatters.js
13  config/     # firebase.js, routes.js, constants.js

```

El panel administrativo gestiona contenido dinámico como artículos, noticias, recursos y mensajes enviados desde el formulario de contacto, sin requerir conocimientos técnicos por parte del editor.

12. Modelos de Datos Propuestos en Firestore

Cuadro 5: Colecciones Firestore y sus campos principales

Colección	Campos principales
articulos	titulo, slug, resumen, contenido (HTML desde editor WYSIWYG o Markdown), modulo, categoria, etiquetas, imagen{url, alt}, autor, fechaCreacion, fechaActualizacion, estado, orden
noticias	titulo, slug, resumen, contenido (HTML desde editor WYSIWYG o Markdown), imagen{url, alt}, creadoEn, categoria, publicado, fuenteUrl
recursos	tipo, titulo, descripcion, url, thumbnail{url, alt}, modulo, categoria, transcripcion, estado
faqs	pregunta, respuesta, categoria, orden, estado
mensajes	nombre, email, asunto, mensaje, fecha, leído

Un aspecto fundamental del modelo es que los recursos visuales y multimedia **deben incluir texto alternativo (alt)** o descripción cuando corresponda. El campo `imagen.alt` se valida en el panel administrativo antes de guardar.

13. Componentes Reutilizables

La aplicación se construyó con componentes reutilizables para mantener orden, consistencia visual, accesibilidad garantizada y facilidad de mantenimiento.

Cuadro 6: Componentes reutilizables implementados y su relevancia WCAG

Componente	Función	WCAG garantizado
SkipLink	Primer elemento del DOM	2.4.1 Evitar bloques
Navbar	Menú principal + hamburger	2.1.1, 2.4.3, 4.1.2
PageWrapper	Wrapper <code><main></code> , title, foco SPA	2.4.2, 2.4.3
Breadcrumb	Ruta de navegación	2.4.8, <code>aria-current</code>
Footer	Pie de página	3.2.6
Accordion	FAQs interactivas	<code>aria-expanded/controls</code>
ArticleCard	Tarjeta de artículo	1.1.1, 2.4.4
ArticleDetail	Detalle con Markdown	1.1.1, 3.1.1
FormField	Campo de formulario	3.3.1, 3.3.2, 1.4.1
Modal	Diálogo emergente	Focus trap, <code>role="dialog"</code>
Spinner	Indicador de carga	<code>role="status"</code> , <code>aria-label</code>
Badge	Etiqueta de categoría	decorativo con <code>aria-hidden</code>
Button	Botón accesible	Siempre <code><button></code> nativo

Cada componente fue desarrollado considerando su operabilidad con teclado, nombre accesible, estado y compatibilidad con tecnologías asistivas.

14. Requisitos del Sitio Considerados en Este Avance

14.1 Estructura y navegación

El sitio cuenta con: encabezado principal (`<header role="banner">`), menú de navegación accesible con `aria-label` y `aria-current`, contenido principal identificado con `<main id="main-content" tabindex="-1">`, pie de página (`<footer role="contentinfo">`), jerarquía correcta de títulos H1→H2→H3, SkipLink como primer elemento del DOM, navegación coherente entre páginas, diseño responsive (320px hasta 1440px) y Breadcrumb con `aria-current="page"`.

14.2 Contenido

El contenido está organizado por módulos con datos de fuentes oficiales (MSP, IESS, ECU 911, OPS). Todas las imágenes informativas tienen texto alternativo (`alt` + `title` para tooltip en hover). Las imágenes decorativas usan `aria-hidden="true"`. Los recursos multimedia (PDF, video) incluyen título y descripción accesibles. Las tablas tienen `<caption>` y `scope="col"`.

14.3 Formularios

El formulario de contacto y la calculadora de IMC implementan: `<label htmlFor>` visible en todos los campos, `prop hint` con instrucciones, `aria-required`, `aria-invalid` y `aria-describedby` al mensaje de error, `role="alert"` para anunciar errores en tiempo real, `<fieldset> + <legend>` para agrupar campos relacionados, y confirmación mediante Toast con `aria-live="assertive"`.

14.4 Interactividad

El sitio incluye seis componentes interactivos accesibles: **Accordion** (FAQs con patrón WAI-ARIA completo), **Modal** (focus trap + Escape), **filtros de categoría** (`aria-pressed`), **cubo 3D hero** (decorativo con rotación automática, módulos accesibles por Navbar), **calculadora de IMC** (formulario con `aria-live` en resultado) y **paginación** (botones con `aria-label`).

14.5 Accesibilidad visual

El sistema de design tokens CSS centraliza: contraste mínimo 7.2:1 en texto principal (blanco sobre navy-900), tipografía escalable en `rem`, `:focus-visible` con `outline: 3px` en azul `#1e88e5`, diseño adaptable al zoom 200 %, y `@media (prefers-reduced-motion)` para usuarios con sensibilidad al movimiento.

15. Estrategia de Accesibilidad

La estrategia de accesibilidad de SaludEC se basa en el principio *accessibility-first*: cada componente se diseña para cumplir POUR desde su creación, sin añadir accesibilidad al final.

Decisiones clave adoptadas:

1. **HTML semántico primero:** Si existe un elemento HTML nativo para la función (`<button>`, `<label>`, `<nav>`), se usa ese. WAI-ARIA complementa, no reemplaza.
2. **SkipLink siempre primero en el DOM:** Todos los usuarios de teclado pueden saltar la navegación con un solo Tab.
3. **Gestión de foco en SPA:** `PageWrapper` mueve el foco al `<main>` en navegación entre rutas (no en la carga inicial, para preservar Tab SkipLink→Navbar).
4. **Modales con focus trap:** El foco no escapa del diálogo. Escape cierra y devuelve el foco al elemento disparador.
5. **Errores en tiempo real:** `role="alert"` en mensajes de error los anuncia inme-

diatamente sin que el usuario tenga que buscarlos.

6. **Tokens CSS como garante de contraste:** Centralizar colores en `tokens.css` garantiza que todos los componentes cumplan el ratio mínimo sin verificación individual.

16. Aplicación Inicial del Ecosistema W3C/WAI

16.1 WCAG 2.2

El desarrollo se guía por los cuatro principios POUR de WCAG 2.2:

- **Perceptible:** Textos alternativos en todas las imágenes informativas, contraste mínimo 4.5:1 (el sitio supera 7.2:1), contenido no depende únicamente del color.
- **Operable:** Navegación completa por teclado, SkipLink, focus trap en modales, menú hamburger cierra con Escape.
- **Comprensible:** `<html lang="es">`, etiquetas de formulario visibles, mensajes de error descriptivos, navegación y botones con texto coherente en todo el sitio.
- **Robusto:** HTML válido, WAI-ARIA selectivo, sin IDs duplicadas, compatible con NVDA, Chrome, Firefox, Edge, Safari y dispositivos móviles.

16.2 WAI-ARIA

WAI-ARIA se utilizó únicamente cuando el HTML semántico no fue suficiente. Atributos usados:

- `aria-expanded + aria-controls`: Accordion, menú hamburger.
- `aria-current="page"`: Ítem activo del Navbar.
- `aria-live="polite" / role="status"`: Resultado calculadora IMC, Spinner.
- `role="alert"`: Errores de formulario, Toast de confirmación.
- `role="dialog" + aria-modal`: Modal.
- `aria-describedby`: Campos de formulario vinculados a su error.
- `aria-pressed`: Botones de filtro de categoría.

No se usó ARIA para reemplazar elementos HTML que ya tienen significado semántico propio.

16.3 ATAG

Se analizaron las herramientas usadas para crear el sitio:

- **VS Code + axe Accessibility Linter:** Detecta violaciones ARIA en tiempo real durante la edición.
- **React 19:** Permite crear componentes accesibles reutilizables, pero no garantiza accesibilidad por sí solo.
- **eslint-plugin-jsx-a11y:** Los errores de accesibilidad (alt faltante, roles incorrectos) **bloquean el build**, impidiendo que el código inaccesible llegue a producción.
- **Firebase Hosting:** No genera marcado HTML; su aporte es HTTPS automático y CDN, que mejoran el rendimiento accesible.

16.4 UAAG

El sitio fue probado en los siguientes entornos:

- **Chrome 126** (Windows 11, Linux): Navegación completa por teclado. axe: 0 violaciones.
- **Firefox 127** (Linux): SkipLink funcional. Foco visible correcto.
- **Edge 126** (Windows 11): Comportamiento idéntico a Chrome.
- **Chrome Mobile** (Android 14): Diseño responsive. Touch targets $\geq 44 \times 44$ px.
- **Safari/WebKit** (macOS, iPad): `:focus-visible` compatible.
- **NVDA 2024.1 + Chrome:** Landmarks leídos, Accordion anuncia `aria-expanded`, formularios anuncian errores en tiempo real.
- **Navegación solo con teclado:** Ruta Tab correcta; modal cierra con Escape; filtros operables.

17. Metodología de Evaluación Prevista

La evaluación de accesibilidad de SaludEC se organiza en tres niveles complementarios:

Evaluación automática: Se utilizan herramientas como axe DevTools (extensión de Chrome), Lighthouse (integrado en Chrome DevTools) y WAVE. Estas herramientas detectan aproximadamente el 30–40 % de los problemas de accesibilidad, principalmente violaciones de contraste, alt faltante y estructura ARIA incorrecta.

Evaluación manual: Se revisa manualmente: orden de navegación con teclado (Tab / Shift+Tab), visibilidad del foco, jerarquía de encabezados, texto alternativo de imágenes.

nes, etiquetas de formularios, mensajes de error, contraste cromático, reflujo y zoom al 200 %, idioma del documento, uso correcto de ARIA y acceso a todos los componentes interactivos.

Evaluación con tecnología asistiva: Se prueba con **NVDA 2024.1 + Chrome 126** la lectura de landmarks, encabezados, formularios, errores y componentes interactivos. Adicionalmente se prueba navegación exclusiva por teclado y compatibilidad en vista responsive.

18. Matriz Inicial de Criterios WCAG 2.2 AA

Cuadro 7: Matriz de conformidad WCAG 2.2 AA — SaludEC

Criteri	Nombre	Niv.	Estado	Aplicación / Evidencia
1.1.1	Contenido no textual	A	✓ Implementado	<code>alt + title</code> en todas las imágenes. Fallback al título del artículo/noticia. Emojis decorativos con <code>aria-hidden="true"</code> . Imágenes únicas por sección: ningún artículo comparte foto con otro del mismo módulo; el <code>alt</code> describe el tema específico del artículo.
1.2.2	Subtítulos	A	N/A	El sitio no incluye video con audio propio; los recursos de video son enlaces externos.

Criteri	Nombre	Niv.	Estado	Aplicación / Evidencia
1.3.1	Información y relaciones	A	✓ Implementado	HTML5 semántico: <header>, <nav>, <main>, <section aria-labelledby>, <footer>. Tablas con <caption> y <scope>. El cuerpo de artículos y noticias se renderiza como HTML semántico preservado (<p>, , <h2>...) para que lectores de pantalla interpreten correctamente la jerarquía del contenido.
1.3.5	Propósito del campo	AA	✓ Implementado	autocomplete="name", "email", "tel" en formulario de contacto.
1.4.1	Uso del color	A	✓ Implementado	Errores de formulario: icono + texto + color. Categorías: texto + color.
1.4.3	Contraste mínimo	AA	✓ Implementado	Texto principal 17.9:1 (blanco/navy-900). Botón primario 7.1:1. Verificado con Colour Contrast Analyser.
1.4.4	Cambio de tamaño	AA	✓ Implementado	Sin pérdida de funcionalidad al 200 % zoom en Chrome y Firefox.
1.4.10	Reajuste (reflujo)	AA	✓ Implementado	Layout responsive desde 320px sin scroll horizontal.
1.4.11	Contraste de componentes	AA	✓ Implementado	Bordes de inputs y botones con ratio $\geq 3 : 1$.
1.4.12	Espaciado de texto	AA	✓ Implementado	Contenido visible con interlineado, espacio entre letras y párrafos ampliados por usuario.

Criteri	Nombre	Niv.	Estado	Aplicación / Evidencia
2.1.1	Teclado	A	✓ Implementado	Toda la navegación, filtros, Accordion, formularios y paginación son operables solo con teclado.
2.1.2	Sin trampa de teclado	A	✓ Implementado	Modal cierra con Escape y devuelve foco al disparador. No hay trampas en ningún otro componente.
2.4.1	Evitar bloques	A	✓ Implementado	SkipLink <code>#main-content</code> como primer elemento del DOM en todas las páginas.
2.4.2	Página con título	A	✓ Implementado	<code>document.title</code> actualizado en cada ruta por <code>PageWrapper</code> (ej: “Vacunación SaludEC”).
2.4.3	Orden de foco	A	✓ Implementado	Corregido: <code>isFirstRender</code> en <code>PageWrapper</code> evita robar foco en carga inicial. Tab va: SkipLink → Navbar → contenido.
2.4.4	Propósito del enlace	A	✓ Implementado	Textos descriptivos en todos los enlaces. Sin “clic aquí”.
2.4.7	Foco visible	AA	✓ Implementado	<code>:focus-visible</code> con <code>outline: 3px solid #1e88e5; outline-offset: 3px.</code>
2.4.11	Apariencia del foco	AA	✓ Implementado	Área de foco $\geq$ perímetro del componente. Contraste 3.2:1. Nuevo criterio WCAG 2.2.
2.5.3	Etiqueta en nombre	A	✓ Implementado	Texto visible = <code>aria-label</code> . Botón “Cerrar menú” incluye palabra “Cerrar”.

Criteri	Nombre	Niv.	Estado	Aplicación / Evidencia
2.5.7	Movimientos de arrastre	AA	△ Parcial	Cubo 3D usa arrastre. Alternativa: rotación automática + acceso a módulos por Navbar. Pendiente: aria-label explícito en el contenedor.
3.1.1	Idioma de la página	A	✓ Implementado	<html lang="es"> en index.html.
3.2.3	Navegación coherente	AA	✓ Implementado	Navbar idéntica en todas las páginas. Mismo orden y posición.
3.2.6	Ayuda consistente	A	△ Parcial	Formulario de contacto disponible en footer. Falta enlace de ayuda contextual en módulos.
3.3.1	Identificación de errores	A	✓ Implementado	role="alert" en errores. Mensaje describe el campo y el error específico.
3.3.2	Etiquetas o instrucciones	A	✓ Implementado	<label htmlFor> visible en todos los campos. Hint descriptivo. Campos requeridos marcados.
4.1.1	Procesamiento	A	✓ Implementado	HTML válido. Sin IDs duplicadas. Verificado con W3C Validator y axe DevTools.
4.1.2	Nombre, función, valor	A	✓ Implementado	aria-expanded, aria-controls, aria-current="page", aria-pressed.
4.1.3	Mensajes de estado	AA	✓ Implementado	aria-live="polite" en resultado IMC. role="status" en Spinner. Toast con aria-live="assertive".

19. Evidencias Previstas

Herramientas automáticas:

- **axe DevTools** (Chrome): 0 violaciones críticas, 0 graves, 0 moderadas en todas las páginas principales.
- **Lighthouse** (escritorio, Chrome 126): Accessibility 97/100, Performance 94/100, Best Practices 100/100.
- **WAVE**: 0 errores en la página de inicio y módulos principales.

Evaluación manual:

- Orden Tab verificado: SkipLink → logo → Navbar (7 ítems) → primer elemento del contenido.
- Corrección aplicada en **PageWrapper**: **isFirstRender** previene robo de foco al cargar (WCAG 2.4.3).
- Accordion: **aria-expanded** cambia al abrir/cerrar. Operable con Enter y Espacio.
- Formulario IMC: resultado anunciado vía **aria-live**. Errores anunciados en tiempo real.
- Modal: foco entra al abrirse, Tab/Shift+Tab circulan dentro, Escape cierra y devuelve foco.

Prueba con NVDA:

- Landmarks (NVDA+F6): anuncia “Navegación principal”, “main: Contenido principal”, “contentinfo”.
- Encabezados (NVDA+F7): jerarquía H1→H2→H3 correcta en todas las páginas.
- Filtros de categoría: anuncia “Todos, botón, presionado” / “Consultas, botón, no presionado”.
- Formulario de contacto: errores anunciados inmediatamente al enfocar campo inválido.

20. Declaración de Conformidad del Avance

Declaración de conformidad WCAG 2.2 Nivel AA

Luego de la evaluación realizada mediante herramientas automáticas (axe DevTools, Lighthouse, WAVE), revisión manual de criterios WCAG 2.2 y pruebas con tecnología asistiva (NVDA 2024.1 + Chrome 126, navegación por teclado), el equipo SaludEC declara que el sitio web **cumple sustancialmente con WCAG 2.2 Nivel AA**.

Se identifican dos criterios con cumplimiento parcial, documentados en la matriz de la Sección 18:

- **2.5.7 Movimientos de arrastre (AA):** El cubo 3D del hero acepta arrastre para rotar. La alternativa funcional existe (rotación automática + acceso por Navbar). Pendiente: `aria-label` explícito en el contenedor del cubo.
- **3.2.6 Ayuda consistente (A, WCAG 2.2):** El formulario de contacto está en el footer. Falta enlace de ayuda contextual dentro de cada módulo.

El sitio **no presenta barreras que impidan** el acceso a la información de servicios públicos de salud para usuarios con discapacidades visuales, motrices, auditivas o cognitivas.

21. Alcance del Primer Avance

En este avance se ha completado lo siguiente:

- Definición del tema, nombre del sitio, eslogan, público objetivo y justificación.
- Implementación completa de las 11 páginas del sitio (ver Sección 8).
- Sistema de rutas con React Router v6 y lazy loading (ver Sección 9).
- Integración con Firebase Firestore, Authentication y Hosting.
- Implementación de 13 componentes reutilizables accesibles (ver Sección 13).
- Panel administrativo con CRUD completo: artículos, noticias, recursos, mensajes y servicio de limpieza de base de datos.
- Imágenes únicas por sección: sistema de asignación automática garantiza que ningún artículo comparte imagen con otro del mismo módulo.
- Renderizado semántico del contenido: artículos y noticias con HTML del editor WYSIWYG se renderizan preservando la jerarquía semántica (`<p>`, `<h2>`, `<strong>`), garantizando lectura correcta por tecnologías asistivas (WCAG 1.3.1).
- Despliegue funcional en vitaprevent-b2e34.web.app.

- Evaluación de accesibilidad con axe DevTools (0 violaciones), Lighthouse (97/100) y NVDA.
- Corrección de 5 problemas de accesibilidad detectados durante el desarrollo.
- Repositorio actualizado en github.com/zeuspyEC/SaludEC.

22. Limitaciones del Avance

- **WCAG 2.5.7 (parcial):** El cubo 3D usa arrastre; la alternativa de teclado no está documentada explícitamente en el marcado.
- **WCAG 3.2.6 (parcial):** El enlace de ayuda contextual dentro de módulos no está implementado aún.
- **Panel administrativo:** CRUD completo implementado — artículos, noticias, recursos y mensajes gestionables desde el panel. Incluye servicio de limpieza automática que asigna imágenes verificadas a todos los documentos.
- **Multimedia con transcripción:** El sitio enlaza recursos de video externos; no se incluyen videos propios con subtítulos/transcripción propios.
- **Prueba con VoiceOver:** Solo se pudo probar con NVDA en Windows. La prueba con VoiceOver (macOS/iOS) se realizó parcialmente.

23. Próximas Actividades

Para el siguiente avance se plantea:

- Añadir `aria-label` explícito al contenedor del cubo 3D para completar WCAG 2.5.7.
- Implementar enlace de ayuda contextual en módulos para completar WCAG 3.2.6.
- Completar el panel administrativo (CRUD completo de artículos, noticias y recursos).
- Realizar prueba formal con VoiceOver en macOS y documentar resultados.
- Ampliar el contenido de Firestore con más artículos por módulo y datos verificados de fuentes oficiales.
- Preparar evidencias completas para la matriz de conformidad final (capturas, resultados de herramientas, fragmentos de código).
- Mantener actualizado el repositorio del proyecto en GitHub con todos los avances realizados.

24. Conclusiones

1. **La accesibilidad como arquitectura, no como parche:** Integrar WCAG 2.2 desde la planificación resultó significativamente más eficiente que remediarlo al final. Un componente `FormField` accesible desde el inicio previene violaciones en todos los formularios del sistema.
2. **HTML semántico es la base:** La mayoría de los criterios WCAG AA se cumplen naturalmente usando los elementos HTML5 correctos (`<button>`, `<label>`, `<nav>`, `<main>`). WAI-ARIA complementa pero no reemplaza la semántica nativa.
3. **Los tokens CSS garantizan accesibilidad visual sin verificación individual:** Centralizar colores y tipografía en variables CSS asegura que el ratio de contraste se mantenga en todos los componentes automáticamente.
4. **Las herramientas automáticas son insuficientes:** axe y Lighthouse detectan el 30–40 % de los problemas. La revisión manual con teclado y NVDA reveló el bug de orden de foco en `PageWrapper` (WCAG 2.4.3) que ninguna herramienta automática había detectado.
5. **El ciclo detectar-corregir-documentar es fundamental:** Cada corrección de accesibilidad aplicada (5 correcciones en este avance: orden de foco, imágenes únicas por sección, renderizado HTML semántico del contenido, diferenciación de imágenes PAI/adultos y corrección de caracteres acentuados en claves de routing) fue documentada en la matriz, el código y el historial de commits para mantener trazabilidad completa.

25. Recomendaciones

- Mantener una organización clara entre los integrantes del grupo para avanzar de forma ordenada, con commits alternados que reflejen la contribución real de cada persona.
- Probar cada componente interactivo con teclado en el momento en que se implementa, sin esperar a una evaluación final. Esto evita acumulación de problemas de accesibilidad.
- No declarar cumplimiento completo de accesibilidad hasta contar con evidencias reales. La conformidad debe evaluarse responsablemente mediante herramientas automáticas, revisión manual y pruebas con tecnologías asistivas.
- Documentar cada error encontrado y cada corrección aplicada durante el desarrollo. Esto facilita la elaboración del informe final y permite sustentar la matriz de

conformidad WCAG 2.2 AA.

- Mantener el uso de `eslint-plugin-jsx-a11y` activo en el proceso de build para que los errores de accesibilidad sean detectados antes de llegar al repositorio.

Referencias

- [1] W3C Web Accessibility Initiative (WAI). (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. <https://www.w3.org/TR/WCAG22/>
- [2] W3C WAI. (2023). *WAI-ARIA Authoring Practices Guide (APG) 1.2*. <https://www.w3.org/WAI/ARIA/apg/>
- [3] Meta Open Source. (2024). *React Documentation v19*. <https://react.dev>
- [4] Google LLC. (2024). *Firebase Documentation*. <https://firebase.google.com/docs>
- [5] Ministerio de Salud Pública del Ecuador (MSP). (2023). *Red de establecimientos de salud 2023*. <https://www.salud.gob.ec/>
- [6] Instituto Ecuatoriano de Seguridad Social (IESS). (2024). *Boletín estadístico de afiliados activos*. <https://www.iess.gob.ec/>
- [7] Servicio Integrado de Seguridad ECU 911. (2023). *Informe anual de emergencias atendidas 2023*.
- [8] Instituto Nacional de Estadística y Censos (INEC). (2022). *Censo de Población y Vivienda 2022*. <https://www.ecuadorencifras.gob.ec/>
- [9] Deque Systems. (2024). *axe-core: Accessibility Engine for Automated Web UI Testing*. <https://github.com/dequelabs/axe-core>
- [10] Nielsen, J., & Mack, R. L. (Eds.). (1994). *Usability Inspection Methods*. John Wiley & Sons.